



MODUL 9 · DATABASE DENGAN PRISMA 7

CRUD: Create, Read, Update, Delete



Operasi database paling fundamental — dari create hingga delete, semuanya type-safe.

CREATE — Insert Data ke Database

create() untuk satu record, createMany() untuk bulk insert.

```
import { db } from '@lib/db'

// — Single create —————
const user = await db.user.create({
  data: {
    email: 'rizki@example.com',
    name: 'Muhammad Rizki',
    role: 'AUTHOR',
  },
  select: {
    // pilih field yang dikembalikan
    id: true,
    name: true,
    email: true,
  },
})
// user: { id: string; name: string; email: string }

// — Create dengan nested relasi —————
const post = await db.post.create({
  data: {
    title: 'Belajar Prisma 7',
    slug: 'belajar-prisma-7',
    content: 'Konten artikel...',
    author: {
      connect: { id: user.id }, // hubungkan ke user yang ada
      // atau:
      // create: { email: "...", name: "...", role: "AUTHOR" } // buat user baru
    },
    include: { author: { select: { name: true } } },
  },
})
```

select vs include

select: pilih field spesifik

```
{
  select: {
    id: true,
    name: true,
    // email tidak → tidak dikembalikan
  }
}
```

include: include relasi

```
{
  include: {
    posts: true // all fields
    // atau:
    posts: {
      select: { title: true }
    }
  }
}
```

Gunakan select untuk performa — hanya fetch yang dibutuhkan.

Hindari include jika tidak perlu relasi.

READ — Query Data dari Database

findMany(), findUnique(), findFirst() — pilih yang sesuai kebutuhan.

```
// — findMany — ambil banyak record —————
const posts = await db.post.findMany({
  where: { published: true, authorId: userId },
  orderBy: { createdAt: 'desc' },
  take: 10, // limit
  skip: 0, // offset (untuk pagination)
  select: { id: true, title: true, slug: true, createdAt: true },
})
// posts: Array<{ id: string; title: string; slug: string; createdAt: Date }>

// — findUnique — ambil satu record by unique field —————
const user = await db.user.findUnique({
  where: { email: 'rizki@example.com' },
  // atau where: { id: "user_id" }
})
// user: User | null — bisa null jika tidak ditemukan!

// — findFirst — ambil record pertama yang cocok —————
const latestPost = await db.post.findFirst({
  where: { authorId: userId, published: true },
  orderBy: { createdAt: 'desc' },
})

// — count — hitung jumlah record —————
const total = await db.post.count({
  where: { published: true },
})
```

UPDATE & DELETE — Mutasi Data

update/upsert untuk update. delete/deleteMany untuk hapus. Selalu hati-hati!

```
// — update — update satu record —————
const updated = await db.post.update({
  where: { id: postId }, // harus unique!
  data: {
    title: 'Judul Baru',
    published: true,
    viewCount: { increment: 1 }, // atomic increment!
    updatedAt: new Date(),
  },
})

// — updateMany — update banyak record —————
const count = await db.post.updateMany({
  where: { authorId: userId },
  data: { published: false },
})
// count.count = jumlah record yang diupdate

// — upsert — update jika ada, create jika tidak —
const profile = await db.profile.upsert({
  where: { userId: userId },
  update: { bio: 'Bio baru' },
  create: { userId: userId, bio: 'Bio pertama' },
})
```

```
// — delete — hapus satu record —————
const deleted = await db.post.delete({
  where: { id: postId },
})

// — deleteMany — hapus banyak —————
const result = await db.post.deleteMany({
  where: {
    authorId: userId,
    published: false,
    createdAt: {
      lt: new Date(Date.now() - 30 * 86400000), // > 30 hari
    },
  },
})
console.log(`Deleted: ${result.count} posts`)

// ⚠ deleteMany TANPA where = hapus SEMUA!
// Prisma tidak akan stop kamu dari ini.
// Selalu pastikan where condition ada!

// — Soft delete pattern (lebih aman) —————
const softDeleted = await db.post.update({
  where: { id: postId },
  data: { deletedAt: new Date() }, // tidak hapus, hanya
  mark
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa perbedaan findUnique dan findFirst?

- A) Tidak ada bedanya
- B) findUnique hanya untuk unique fields (@id, @unique), findFirst bisa untuk kondisi apapun
- C) findFirst selalu lebih cepat

Q2 — Bagaimana cara increment viewCount secara atomic di Prisma?

- A) data: { viewCount: user.viewCount + 1 } (baca dulu, tambah, update)
- B) data: { viewCount: { increment: 1 } } — atomic, aman dari race condition
- C) Tidak bisa atomic di Prisma

Q3 — Apa risiko dari db.post.deleteMany() tanpa where clause?

- A) Prisma akan throw error
- B) Menghapus SEMUA records di tabel — tidak ada konfirmasi dari Prisma
- C) Hanya menghapus 100 records per default

Tugas Mandiri

Implementasi CRUD lengkap untuk Post model.

01 Create & Read

Buat Server Actions: `createPost(data)` dan `getPostBySlug(slug)`. Test `createPost` di form, `getPostBySlug` di page detail.

02 Update

Buat `updatePost(id, data)` dengan `update`. Tambahkan `atomic increment viewCount` setiap kali post detail dibuka.

03 Soft Delete

Tambahkan field `deletedAt: DateTime?` ke Post model. Buat `softDeletePost(id)` yang `update deletedAt`. Tambahkan filter `{ where: { deletedAt: null } }` di semua query.

04 Upsert

Implementasikan `upsert` untuk UserPreferences: jika belum ada buat baru, jika sudah ada `update`. Test idempotency: run dua kali, hasilnya sama.



Atomic operations: `{ increment: 1 }`, `{ decrement: 1 }`, `{ multiply: 2 }` — gunakan untuk counter agar aman dari race condition.

Yang Sudah Kita Pelajari

- ✓ `create()` + `createMany()` untuk insert. Nested `create/connect` untuk data relasi.
- ✓ `findUnique` (unique fields) vs `findFirst` (kondisi apapun). Return null jika tidak ditemukan!
- ✓ `{ increment: 1 }` untuk atomic counter — lebih aman dari read-then-write.
- ✓ `deleteMany` tanpa `where` = hapus SEMUA. Selalu sertakan `where` clause.
- ✓ Soft delete: update `deletedAt` daripada hapus — lebih aman, bisa di-restore.

→ Selanjutnya: **Bab 7 — Filtering, Sorting, Pagination & Transactions**

